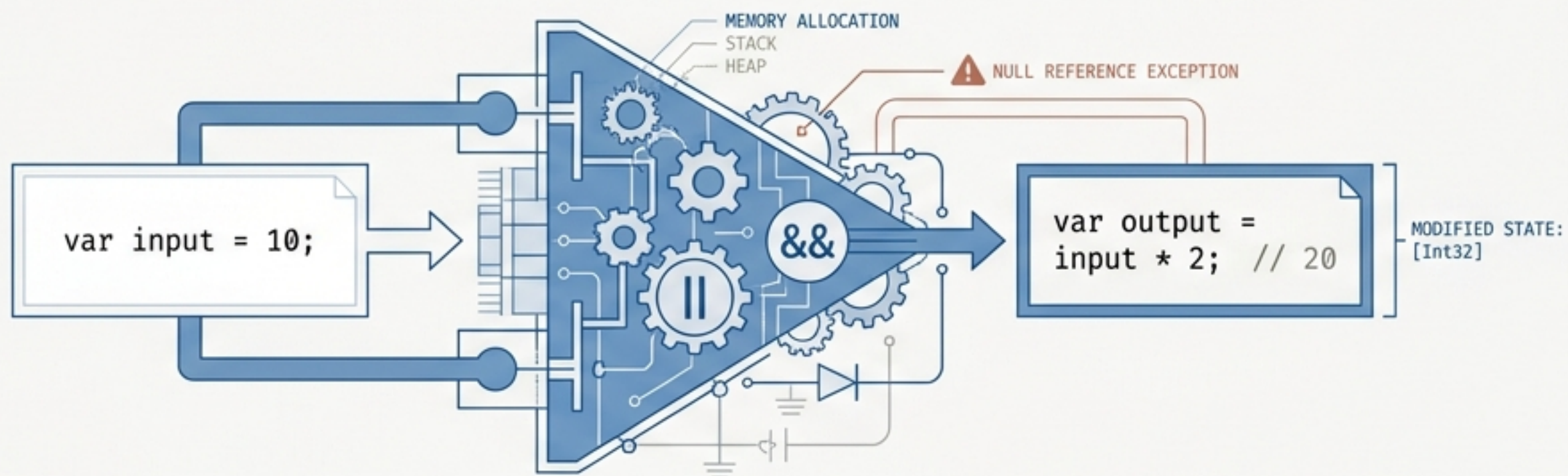


C# 기본 문법: 데이터와 연산자의 설계도

변수의 해부학부터 메모리의 한계, 그리고 연산자의 작동 원리까지



데이터 타입의 해부학 (DATA TYPES)

TYPE	MEMORY (STACK/HEAP)
int	4 bytes (STACK)
bool	1 byte (STACK)
string	Reference (HEAP)
object	Reference (HEAP)

연산자의 작동 원리 (OPERATORS)

OPERATOR	FUNCTION
<code>++</code> / <code>--</code>	Increment/Decrement
<code>+</code> / <code>-</code> / <code>*</code> / <code>/</code>	Arithmetic
<code>==</code> / <code>!=</code> / <code>></code> / <code><</code>	Comparison
<code>&&</code> / <code> </code> / <code>!</code>	Logical

변수의 해부학: 데이터를 담는 식별자

변수는 두 단계를 거쳐 완성됩니다: 1. 선언 (공간 확보) → 2. 할당 (값 주입).

자료형 (Type)

(어떤 형태의 데이터를 담을지 결정)

이름 (Identifier)

(메모리 공간의 이름표)

값 (Value)

(실제 저장되는 데이터)

`double pi = 3.14159265;`



1. 선언: 빈 컨테이너 상자 생성



2. 할당: 상자 안에 데이터를 주입

정수 자료형과 메모리의 물리적 한계

C#에서 정수는 무한하지 않습니다.
할당된 바이트(Byte) 크기에 따라
담을 수 있는 한계값이 존재합니다.

int:	4바이트 (가장 일반적으로 사용)
long:	8바이트



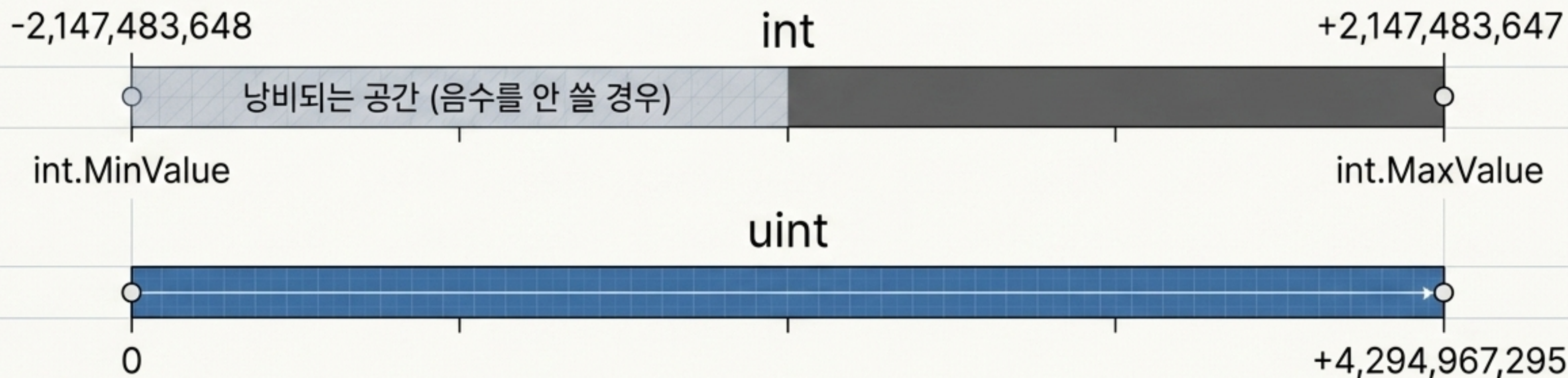
```
01 static void Main(string[] args)
02 {
03     int a = 2147483640;
04     int b = 52273;
05     Console.WriteLine(a + b);
06 }
```

실행 결과
-2147431383

오버플로(Overflow) 발생!
허용된 최댓값을 넘으면 최솟값으로 순환합니다.

한계 극복: Unsigned 자료형의 공간 재활용

음수를 사용하지 않는다면, 버려지는 음수 영역을 양수 영역으로 끌어와 최댓값을 2배로 늘릴 수 있습니다.



```
uint unsignedInt = 4147483647;  
ulong unsignedLong = 11223372036854775808;
```

Pro-Tip | 부호가 없는(unsigned) 자료형은 기존 키워드 앞에 'u'를 붙여 사용합니다 (uint, ulong).

실수(Real Numbers)와 불리언(Booleans)

정밀도의 차이 (실수 자료형)

키워드	설명
float	4바이트의 실수
double	8바이트의 실수

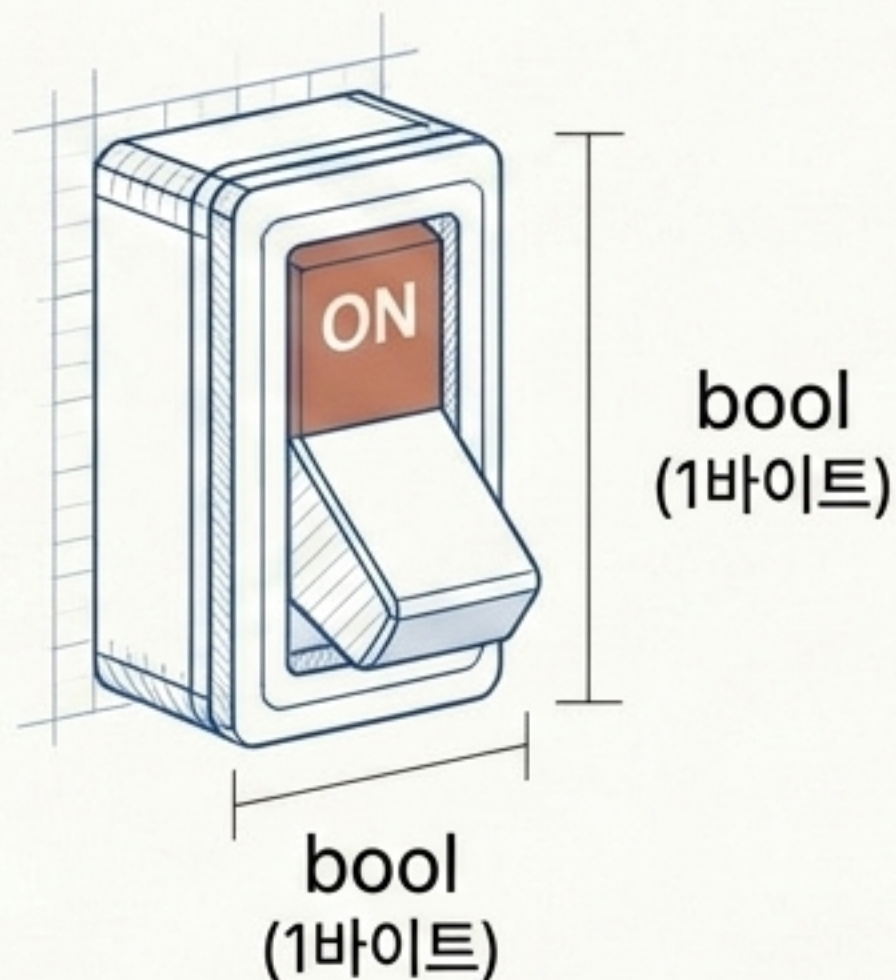
float
(4바이트)

double
(8바이트)



```
double a = 52.273;  
double b = 103.32;
```

논리의 스위치 (불 자료형)

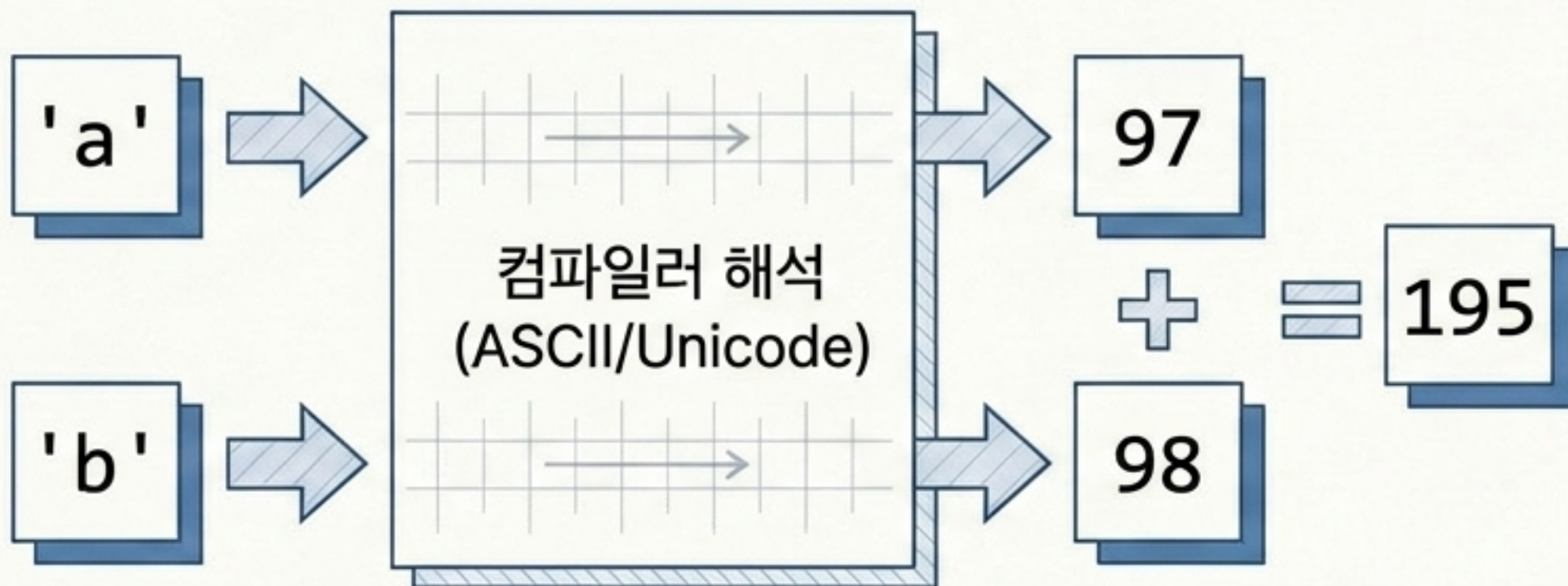


```
bool one = 10 < 0;  
bool other = 20 > 10;
```

문자(char) 자료형의 이중성

컴퓨터는 문자를 모릅니다. char 자료형(2바이트)은 사실 내부적으로 숫자로 처리됩니다.

The Character-Integer Bridge



```
01 static void Main(string[] args)
02 {
03     char a = 'a';
04     char b = 'b';
05
06     Console.WriteLine(a + b);
07 }
```

실행 결과

195

핵심: 문자는 문자열(string)보다 정수(int) 자료형에 더 가깝습니다.

컴파일 오류의 비밀: 문자열과 Sizeof의 예외

Problem Block

~~sizeof(string)~~ 컴파일 오류

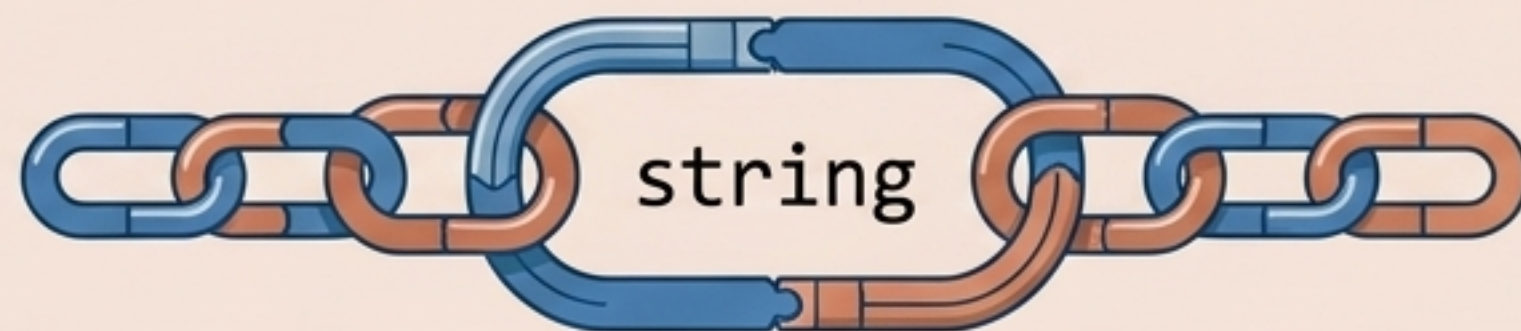
기본 자료형 (컴파일 타임 측정 가능)



 readonly struct System.Int32
Represents a 32-bit signed integer.

크기가 고정된 구조체 (Struct) 형태.
sizeof 사용 가능.

문자열 (런타임 동적 할당)



길이가 변할 수 있는 클래스 (Class) 형태
(System.String).

메모리 크기가 유동적이어서 **sizeof**로 측정 불가!

C# 기본 자료형 총정리 (The Master Matrix)

키워드 (Keyword)	크기 (Size)	원형 (Base Struct/Class)	데이터 예시 (Format)
int	4바이트	System.Int32	10
long	8바이트	System.Int64	20L
float	4바이트	System.Single	3.14f
double	8바이트	System.Double	3.14
char	2바이트	System.Char	'A'
bool	1바이트	System.Boolean	true
string	가변적 (Class)	System.String	"Hello"

복합 대입 연산자: 코드의 압축과 효율

기본 연산자와 대입 연산자(=)를 결합하여 코드를 간결하게 만듭니다.

Transformation Box

Before

```
output = output + 52;
```



압축 (Syntactic Sugar)

After

```
output += 52;
```

기본 연산자	복합 대입 연산자
+	+=
-	-=
*	*=
/	%=

문자열 결합 (String Application)

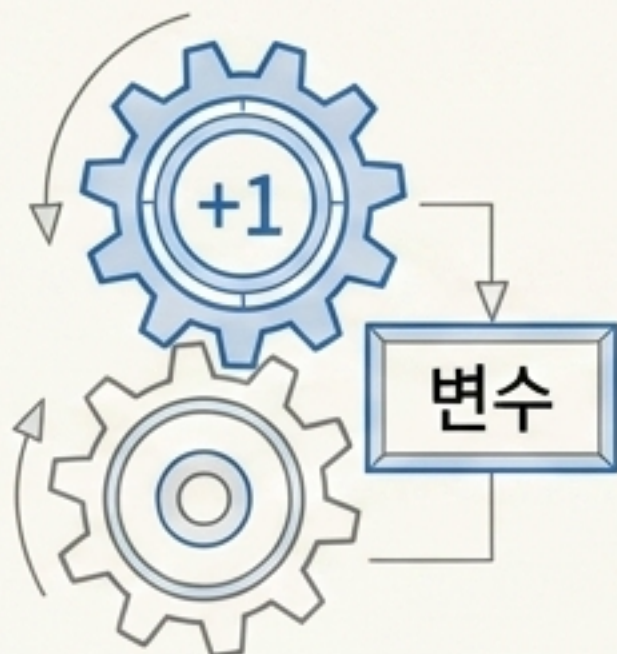
```
"hello " += "world "
```

```
"hello world "
```

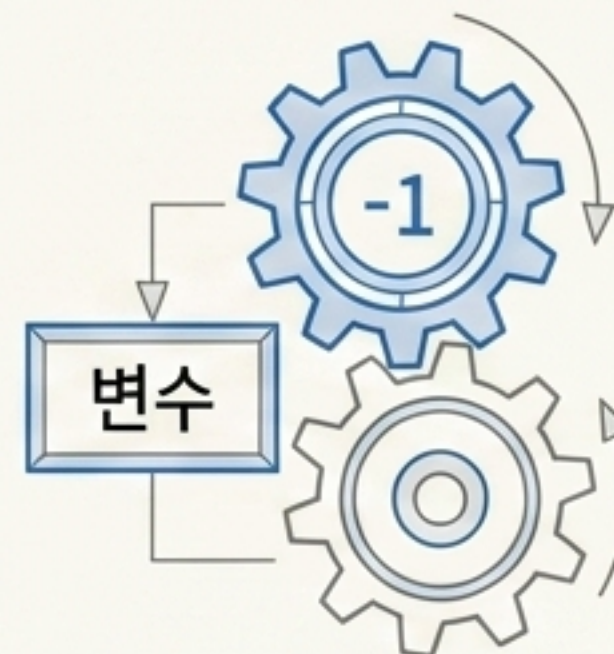
```
output += "world ";
```

증감 연산자: +1과 -1의 톱니바퀴

단항 연산자(Unary Operator)로, 변수의 값을 정확히 1씩 증가시키거나 감소시킵니다.



연산자	설명
[변수]++ 기존 값에 1을 더함 - 후위	++[변수] 기존 값에 1을 더함 - 전위
[변수]-- 기존 값에서 1을 뺌 - 후위	--[변수] 기존 값에서 1을 뺌 - 전위



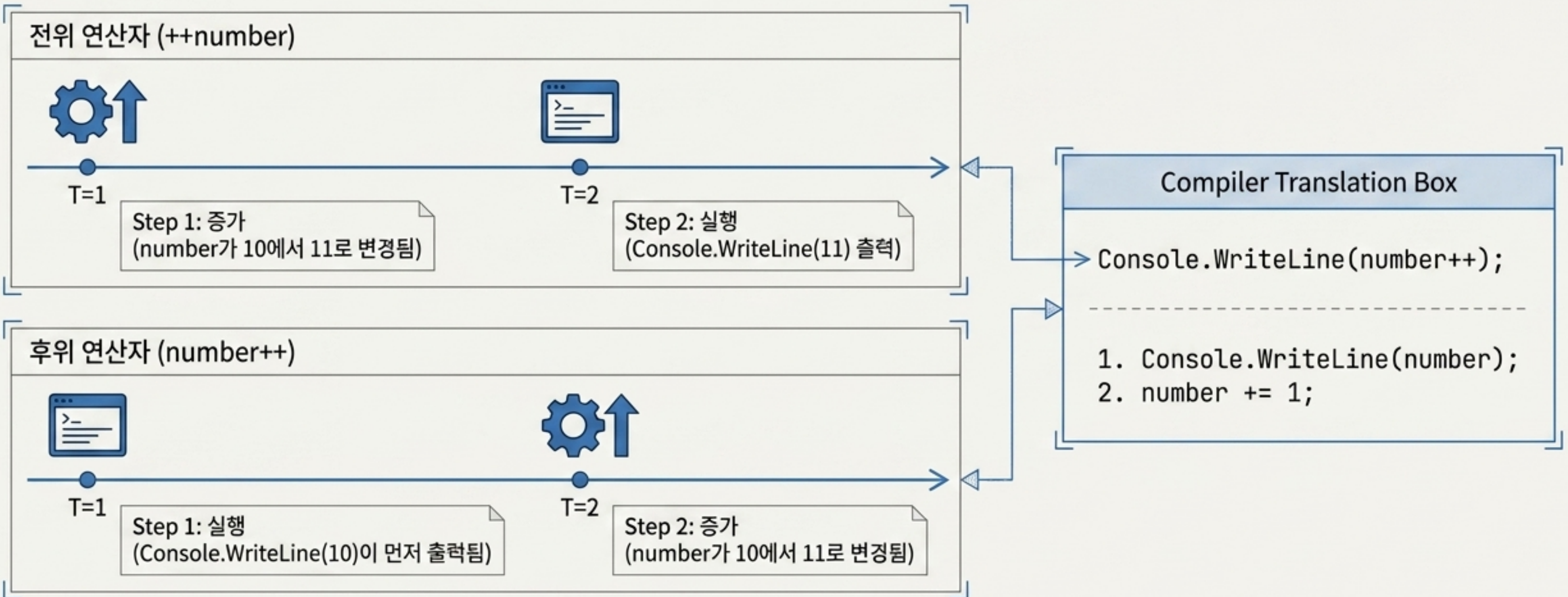
```
01 int number = 10;  
02 number++;  
03 Console.WriteLine(number);  
04  
05 number--;  
06 Console.WriteLine(number);
```

```
11  
10
```

독립적인 한 줄에서 사용할 때는 전위와 후위의 결과가 동일합니다.

전위(Prefix) vs 후위(Postfix): 실행의 타임라인

다른 명령어와 함께 쓰일 때, 증감 연산자의 위치는 실행 '순서'를 결정합니다.



Best Practice: 코드의 가독성을 지키는 원칙

코드는 똑똑해 보이는 것보다 읽기 쉬운 것이 훨씬 중요합니다.
증감 연산자는 독립적인 줄에 사용하는 것을 권장합니다.

⚠ 혼란스러운 코드 (Bad) ⚠

```
Console.WriteLine(number++);  
Console.WriteLine(++number);
```

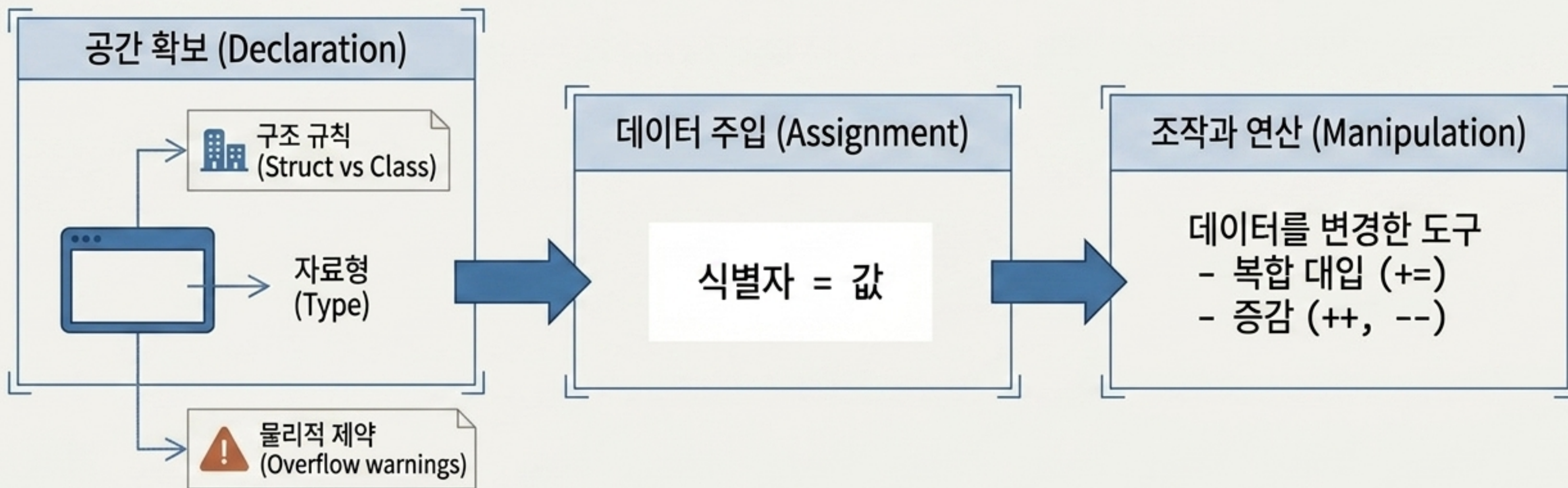
사람이 머릿속으로 상태를 추적해야 하므로
버그 발생률이 높습니다.

✅ 명확한 코드 (Good) ✅

```
number++;  
Console.WriteLine(number);
```

연산과 출력을 분리하여
코드의 의도가 즉시 파악됩니다.

C# 기본 설계도 (The Architecture of C# Basics)



완벽한 소프트웨어는 데이터의 한계를 이해하고, 적절한 연산자를 선택하는 것에서 시작됩니다.